Instrucciones para la Actividad 3: Investigación sobre GitHub y Trabajo en Ramas

En esta actividad, realizarás una investigación sobre el uso de GitHub y el trabajo en ramas. El objetivo es que entiendas los conceptos fundamentales y cómo aplicarlos en la gestión de proyectos de desarrollo de software. A continuación, te proporcionamos 10 preguntas que deberás responder basándote en tus lecturas, videos, o recursos disponibles en línea.

Instrucciones:

1. Investiga los conceptos clave relacionados con GitHub y el trabajo en ramas.
2. Responde las 10 preguntas proporcionadas de manera clara y concisa, utilizando ejemplos si es necesario.
3. Las respuestas deberán entregarse en un documento, ya sea en formato PDF o Word, y subirse a la plataforma indicada por el instructor.
4. Sé detallado en tus respuestas, asegurándote de que sean comprensibles para alguien que no está familiarizado con GitHub o el trabajo en ramas.

Recursos sugeridos para la investigación:

- Documentación oficial de Git: https://git-scm.com/doc
- Documentación oficial de GitHub: https://docs.github.com
- Videos tutoriales de GitHub en YouTube
- Artículos en blogs o tutoriales sobre Git y GitHub

Preguntas sobre GitHub y Trabajo en Ramas:

1. **¿Qué es Git y cuál es su relación con GitHub?**
    - .Git es un sistema de control de versiones es decir una herramienta que te permite rastrear los cambios de un código a lo largo del tiempo, té permite también crear ramas para experimentar sin romper nada, y volver atrás si algo sale mal.
      Git es la herramienta y GitHub es el servicio que guarda tus repositorios en línea.
2. **¿Qué es una rama (Branch) en Git y por qué es importante en el control de versiones?**
    - Una rama es simplemente un puntero móvil a un commit específico dentro del historial de tu proyecto. Son importantes ya que te permite desarrollar una funcionalidad nueva, corregir un bug o experimentar sin romper el código estable.
3. **¿Cómo se crea una nueva rama en Git? Escribe el comando y explica su uso.**
    - Existen dos formas principales de crear una rama en Git, dependiendo de si deseas movernos inmediatamente a ella.

    Crear y cambiar de rama simultáneamente
    - El método más común utiliza el comando moderno git switch o el clásico git checkout con la bandera -b. Esto crea la nueva rama basada en el commit actual y cambia el puntero HEAD hacia ella en un solo paso.
      git switch -c nombre-de-la-rama
      # O alternativamente:
      git checkout -b nombre-de-la-rama

    Crear la rama sin cambiar de contexto
    - Si prefieres crear la rama pero seguir trabajando en la rama actual, utiliza el comando git Branch seguido del nombre deseado. Posteriormente, deberás usar git switch o git checkout para moverte a la nueva rama.
      git Branch nombre-de-la-rama
      Para crear una rama basada en una rama específica (no en la actual), especifica la rama de origen al final del comando:
      git switch -c nueva-rama origen/main
4. **¿Cuál es la diferencia entre `git merge` y `git rebase`?**
    - Tanto git merge como git rebase sirven para integrar cambios de una rama a otra, pero lo hacen de formas muy distintas.
    - Git merge conserva el historial original, crea un commit de fusión (merge) que une las dos líneas de historia. El historial muestra que dos ramas se juntaron con todos sus commits originales e intactos.
    - Git rebase reescribe el historial, toma todos los commits de una rama y los coloca encima de la punta del main, como si hubieras empezado a trabajar después. El resultado es un historial lineal y limpio sin commits de fusión. La desventaja es que reescribe los commits.
5. **¿Qué es el comando `git checkout` y cómo se utiliza para cambiar entre ramas?**
    - El comando git checkout permite cambiar de rama, revisar commits específicos y restaurar archivos, aunque en Git moderno (2.23+) se

recomienda usar git switch para cambios de rama y git restore para restaurar archivos.
    - Para moverse a una rama existente, usa: git checkout <nombre-de-la-rama>
    - Para crear una nueva rama y cambiar a ella simultáneamente, utiliza: git checkout -b <nueva-rama>
6. **¿Cómo puedes visualizar todas las ramas disponibles en un repositorio de Git?**
    - Para visualizar todas las ramas disponibles en un repositorio de git se usa el comando git Branch, muestra las ramas locales con un asterisco, marcando la rama en la que se está actualmente.
    - Git Branch: Muestra ramas locales
    - Git Branch -a: Ramas locales y remotas
    - Git Branch -r: Solo ramas remotas
    - Git Branch -v: Muestra ramas junto con su ultimo commit
7. **¿Qué es un Pull Request (PR) en GitHub y cómo se utiliza en el flujo de trabajo colaborativo?**
    - Es una solicitud de revisión; le pides a tu equipo que revise los cambios de tu rama antes de fusionarlos con la rama principal.
    - Actúa como un filtro de calidad, evita que subas códigos con errores y rompas el flujo de trabajo.
    - El flujo usual es:
      Crear la rama.
      Subir la rama a GitHub y abres pull request explicando que se hizo
      El equipo revisa el código y deja comentarios para realizar correcciones
      Se aprueban los cambios y se fusiona con el proyecto principal
8. **¿Qué sucede cuando fusionas (merge) una rama en la rama principal (main o master)?**
    - Fusionar integra el trabajo de la rama principal, crea un commit que une ambas historias mientras que no haya conflicto entre ellas y deja un main como el punto más reciente del proyecto.
    - Los pasos son:
      Integrar cambios.
      Se crea un commit de fusión
      Si ambas ramas reescriben la misma línea de código, se hace una decisión de cual mantener y resuelves el conflicto con git add
      La rama fusionada no se borra, sigue existiendo con todo su historial
      El main queda actualizado con todo el trabajo de la rama
9. **¿Cómo puedes resolver un conflicto de fusión (merge conflict) en Git?**
    - Un conflicto de fusión ocurre cuando dos personas modifican la misma línea del mismo archivo en diferentes ramas o cuando alguien borra un archivo que alguien más estaba editando.
    - Para resolverlo se usan tres pasos:
      Identificar el problema: Cuando intentas hacer un git merge o actualizar la rama, git avisa que hay un conflicto. Para ver exactamente que archivos están afectados, se ejecuta un git status.
      Limpiar el código; Abre el archivo afectado en el editor de código. Veras que git marca la zona del conflicto con símbolos. Lo que se debe de hacer es borrar los símbolos y dejar únicamente el código final que debe quedarse.

Al final, solo se guarda y termina con un git add, git commit y push origin.

10. **¿Qué es el comando `git branch -d` y cuándo se debe utilizar?**
    - Es el comando para borrar una rama local de forma segura; elimina la rama indicada, pero solo si ya ha sido fusionada con otra rama. Git verifica que los commits de esa rama ya estén integrados en otra; si es así, la borra sin problemas.

    - Se debe de utilizar después de fusionar una rama con el main y se quiere limpiar el repositorio. Cuando se termine una funcionalidad y ya no se necesita la rama.